City-Wide Parking Spot Finder System Design for Scalability and

Reliability Date: 2025-08-10 UTC

City-Wide Parking Spot Finder System Design for Scalability and
Reliability Date: 2025-08-10 UTC
One Pager Summary Goal Build a production ready platform that ingests real time parking availability and pricing from sensors, garages, curbside meters and partner APIs, then serves low latency queries for drivers and operators. The system must be highly reliable, horizontally scalable, secure, and compliant. Target experience is sub 150 ms p95 search latency, fresh occupancy within seconds, and safe operations at city scale.

Key Outcomes

Fast and accurate availability search by location and constraints Highly reliable ingestion and storage layers with back pressure and replay Operator dashboards, analytics, and automated pricing integrations Tenant aware platform for multiple cities and operators with strong isolation Strong observability and safety with canary deployments and automatic rollback

Primary Assumptions

City size: 2 million population, peak 400 thousand daily active drivers, 50 thousand managed off street spots, 100 thousand curb spots, 10 thousand sensors and meters emitting every 5 seconds Event rate: 10 thousand to 30 thousand updates per second at peak including partner feeds API load: 8 thousand to 15 thousand read requests per second peak, 2 thousand writes per second peak Freshness target: under 3 seconds end to end from device to query result Two clouds possible. The reference stack cites AWS with cloud agnostic equivalents noted where helpful Mobile clients use HTTPS with HTTP 2 or HTTP 3 and TLS 1.3 Mapping uses vector tiles. No street level video ingested. License plates never stored unless an opt in reservation is made

Functional Requirements

Search available spots near a point or route with filters such as price, ev charging, hours, covered, accessible Turn by turn compatible result ordering and predictions of dwell time and probability of availability Reservations, check in, and release flows with optional payments and QR validation Operator portal for inventory, pricing rules, blackouts, events, and alerts Partner ingestion connectors for garages, meters, sensors, and city data Realtime occupancy and pricing updates pushed to caches and subscribers Anonymized analytics and reports on utilization, turnover, violations, and revenue Tenant separation for multiple cities and operators
Non
Availability target: 99.95 percent for public read APIs, 99.9 percent for writes Latency p95: search 150 ms from edge, reservation create 300 ms, ingestion E2E under 3 seconds Durability: hot data RPO 0 with multi AZ, cold data RPO under 5 minutes Scalability: horizontal scaling on ingestion, processing, caches, and query Security: encryption in transit and at rest, least privilege, audited changes, WAF and rate limiting Compliance: SOC 2 Type 2, GDPR, CCPA with data minimization, DPA, deletion and export

Architecture Overview

Ingestion

Layer Edge collectors receive device and partner webhooks or MQTT messages API Gateway authenticates, applies rate limits, schema validation, and queues payloads Streaming bus such as Kafka or Kinesis partitions by geohash prefix plus tenant id to guarantee ordering per location while enabling parallelism Dead letter topics and exactly once like processing via idempotency keys Schema registry for versioned protobuf or JSON schema contracts
Edge to gateway under 50 ms p95 in region Gateway to Kafka

produce under 20 ms p95 Processing and hot writes under 800 ms p95 End to end ingest to visible state under 3 seconds p95 Query CDN and TLS under 30 ms p95 Cache plus hydrate and rank under 100 ms p95 Response body under 20 ms p95 Public Read API availability 99.95 percent, Write API 99.9 percent, 99.95 percent
Security and Compliance PII minimized. Store user pseudonymous id and tokenized payment id only Encryption TLS 1.3 in transit. KMS managed keys for at rest, per tenant keys where required Access controls RBAC and ABAC, scoped OAuth tokens, service mesh mTLS, signed partner webhooks Privacy Consent flows, data retention and deletion APIs, subject access requests, audit logging and IP anonymization Compliance SOC 2 Type 2 with change management and monitoring, GDPR and CCPA with DPA and subprocessor list WAF, bot mitigation, rate limits and anomaly detection at the edge

Multi Tenancy Strategy Tenant id propagated through every path Logical isolation with tenant id in keys and row level security in Postgres Optional hard isolation by dedicated namespaces and VPCs per premium tenant Per tenant rate limits, keys, encryption, budgets, and dashboards
CI CD and IaC Everything as code with Terraform or Pulumi and Helm for Kubernetes Multi stage pipelines with unit, integration, and canary stages Argo Rollouts for progressive delivery with automatic rollback on SLO burn and error budgets Database migrations through versioned change sets and online DDL Synthetic checks and smoke tests post deploy

Stream Processing

Flink or Kafka Streams jobs build the live occupancy state machine per spot
Debounce and reconcile conflicting readings, apply business rules, and compute probability of availability Compute near real time aggregates by geohash and block face Publish derived states to Redis and to an OLTP store for reservation correctness Fan out to time series storage and to the data lake

Storage Strategy

Hot path Redis Cluster for read heavy geospatial lookups and top N by geohash buckets
Primary OLTP for correctness. Option A: Postgres with PostGIS and partitioning by tenant and geohash. Option B: DynamoDB with a composite key tenant id plus geohash and a spatial index service for radius queries Time series TimescaleDB or Amazon Timestream for historical occupancy and pricing Analytics lake Object storage such as S3 in open formats like Parquet with Glue or Iceberg, queried by Athena or Snowflake Search Elasticsearch or OpenSearch for text and faceted queries on addresses, garages, and policies
Query Layer Edge CDN plus API Gateway terminate TLS and validate tokens Read path queries Redis for candidate spot ids by geohash and filters, then hydrates details from OLTP with read replicas Route aware search optionally uses a precomputed contraction hierarchy or OSRM service to rank results along the route Write path for reservations uses optimistic locking and a short lived distributed lock on the spot id Websockets or Server Sent Events push updates to clients and operator UIs
Data Flow Device or partner sends update to Gateway Gateway enqueues to Kafka topic keyed by tenant and geohash Stream processor reconciles state and writes hot state to Redis and OLTP, appends timeseries, publishes events Clients query API, which fanouts to Redis then OLTP replicas and returns ranked results
Technology Stack Gateway and edge CloudFront or Cloud CDN, API Gateway, WAF, OAuth 2.1 or OpenID Connect Streaming Kafka or Kinesis, Schema Registry, Kafka Connect for sources and sinks Processing Apache Flink or Kafka Streams, Debezium for CDC Data Redis Cluster, Postgres with PostGIS and read replicas or DynamoDB, TimescaleDB or Timestream, S3 or GCS with Iceberg, Athena or BigQuery or Snowflake, OpenSearch Services Kubernetes, gRPC between services, REST for public APIs, GraphQL for client aggregation CI and CD GitHub Actions or GitLab CI, Argo CD and Argo Rollouts or Spinnaker Observability OpenTelemetry, Prometheus and Grafana, Loki, Jaeger, CloudWatch
Data Model Outline Core tables or items Spot id, tenant id, type, location geohash and geometry, attributes, status Lot or Garage id, name, address, operator id, pricing policy id SensorReading id, spot id, timestamp, reading type, value, source Reservation id, user id pseudonymous, spot id, status, start, end, payment id PricingPolicy id, rules, dynamic adjustments, effective dates Event id, type, scope, blackout windows AuditLog id, actor id, action, target id, diff, timestamp Partitioning by tenant id and geohash,

secondary indexes on availability and attributes

Latency Budgets and SLAs

Key Algorithms and Workflows

Availability estimation Simple state machine based on last reading, with exponential decay and confidence score to tolerate missing data Route ranking Distance bucket by geohash and route polyline, then call a routing microservice for final score Reservation Check Redis hot state and OLTP transaction to prevent double booking, with a 30 second hold token and TTL in Redis Dynamic pricing Flink job updates price modifiers by utilization percentiles and lead time buckets

APIs and Integrations

Public REST endpoints GET v1 search with query parameters for point, radius, filters, paging GET v1 routesearch with polyline and filters POST v1 reservations to hold or book a spot PATCH v1 reservations id to confirm or release Websocket channel v1 updates for area topics Partner ingestion POST v1 partner occupancy and pricing with HMAC signed requests MQTT topics for devices by tenant and geohash Payments Integration via PCI compliant provider with tokenization and webhooks Maps and routing Vector tile server and OSRM or provider routing API

Observability and Operations

Golden signals and SLOs defined per service Traces sampled at 5 to 20 percent on hot paths Structured logs with request ids, tenant ids, and PII scrubbing Dashboards for ingestion lag, cache hit, tail latency, and error budgets Oncall runbooks, playbooks, and documented auto remediation policies Cost guards with budget alerts and query quotas
Capacity Planning and Cost Estimate Assume AWS in us west 2 and one secondary region Kafka three brokers at m5.4xlarge and EKS worker pools with autoscaling Redis cluster three shards r6g large Postgres or Aurora with writer r6g large and two readers r6g large Timeseries small cluster S3 data lake with 10 terabytes at rest and 2 terabytes monthly scan Rough monthly cost order of magnitude between 35 thousand and 70 thousand depending on partner traffic and analytics usage Scale levers are shard count, read replicas, and autoscaling policies

Testing Strategy and Failure Drills

Unit and property based tests for geospatial and ranking logic Contract tests for partner connectors and public APIs with schema registry Load, soak, and chaos tests that inject partitions, broker loss, cache cold starts, and failover Game days every quarter that simulate region outage and rollback Disaster recovery rehearsals with restore validation for hot OLTP and cold lake

Phased Build Plan

MVP three months Search within radius using Redis and Postgres, single city, partner webhook and batch CSV connectors, basic operator UI, payments optional Phase two six to nine months Add route aware ranking, dynamic pricing, multi tenant controls, canary deployments, observability maturity, and reservation flows Phase three twelve months Cross city, advanced analytics and lakehouse, hard tenant isolation options, automated demand based pricing, SLA bump to 99.95 percent
Key Choices, Trade Offs, Risks, Mitigations Redis plus OLTP balance speed and correctness. Trade off is cache invalidation complexity mitigated with stream driven updates and short TTLs Kafka introduces ops overhead but provides replay and isolation. Mitigated by managed services Postgres with PostGIS offers rich geospatial while DynamoDB scales without ops. Choose based on team skills and required query patterns Privacy risk from location data is mitigated by aggregation, hashing, and retention limits Partner feed quality variability is mitigated with reconciliation, confidence scores, and manual overrides Vendor lock in is mitigated with open formats such as Parquet and Terraform for IaC
Appendix Latency Table p95 targets in milliseconds Edge TLS 30 Redis read 5 OLTP read 20 Routing score 40 Serialization 10 Total search budget 150 Ingest produce 20 Stream processing 600 to 800 Cache update 50 End to end ingest 3000

City-Wide Parking Spot Finder System Design v2 — Multi-Metro
Growth, Heavy Event Readiness, and Stricter SLOs Date:
2025-08-10 UTC
Overview This v2 update hardens the original design for multi-metro scalability, heavy event surges, and tighter Service Level Objectives (SLOs) while retaining the proven ingestion, storage, and query architecture from v1.
1) Cell (Metro) Architecture - Per-metro “cell” = isolated Kafka/Flink, Redis, OLTP, API tier. - Global control plane handles auth, tenant config, billing, onboarding, and cross-cell observability. - Anycast + geo-routing to nearest healthy cell, with graceful cell failover (read-only mode on failover). - Blast radius limited to a single metro.
2) Stricter SLOs - Search: p95 ≤ 120 ms / p99 ≤ 200 ms. - Reservation: p95 ≤ 250 ms / p99 ≤ 400 ms. - Ingest freshness: p95 ≤ 2.0 s. - Levers: edge tiles (hot data in Edge KV), adaptive TTLs by area heat, SLO-driven autoscaling, priority lanes for reservations.
3) Event Surge Readiness - Event model for predeclared surge windows. - Pre-warm caches, precompute ranked results for routes. - Adaptive geohash partitioning for hotspots. - Debounce windows for sensor updates (250–500 ms) to reduce write amplification. - Graceful degradation to tile-only mode when OLTP is under load.
4) Reservation Consistency - OLTP as source of truth + short-lived Redis leases for spot IDs. - Outbox + transactional messaging for consistent cache/stream updates. - Sagas for hold→pay→confirm with compensating actions and timeouts. - Single-writer ownership of spots to prevent race conditions.
5) Storage & Indexing at Scale - Option A: Postgres + PostGIS per cell with partitioning; multiple read replicas. - Option B: DynamoDB/Scylla with geospatial sidecar service (S2/GeoHash index). - Redis stores only minimal structs for hot reads.
6) ML-Assisted Availability - Feature stream for weather/events/time-of-day. - Real-time inference service with strict time budget; deterministic fallback.
7) Privacy & Multi-Metro Data - Per-cell encryption keys; optional per-tenant keys. - Raw location retention ≤ 14 days; aggregated metrics ≥ 13 months. - Cross-cell analytics via Iceberg tables with differential privacy.
8) Observability & Operations - Tail-based sampling for p95+ traces. - Golden signals per cell (ingestion lag, cache hit, reservation conflicts). - SLO-aware autoscaling for Redis, Kafka, and Flink. - Chaos drills for broker loss, cold cache, AZ evacuation, read-only failover.

9) Capacity Guardrails (Large Metro) - Kafka: 6–9 brokers, 96–192 partitions, replication 3. - Flink: parallelism 64–128; checkpoint ≤ 5 s. - Redis: 6–12 shards, < 65% memory usage. - OLTP: 1 writer + 3–5 readers; partitioned tables. - Edge cache: downtown tiles refreshed every 1–2 s.
10) CI/CD & Safety Nets - Argo Rollouts with canary gating on p95 latency, ingestion lag, error rate. - Schema evolution with backward compatibility enforced via Schema Registry. - Online DDL with shadow writes and dual-read windows.
11) Failure Playbooks - Cache cold start → edge tiles + last-known results. - Redis shard loss → hydrate-heavy mode with reduced result size. - Kafka lag ↑ → widen debounce windows; prioritize reservations. - OLTP backpressure → token-based holds only.
12) Cost Controls - Keep Redis payloads < 200 bytes. - Iceberg compaction & query budgets for Athena/BigQuery. - Event-based pre/post-scaling to avoid 24/7 overprovisioning.
Key Improvements Over v1 - Explicit multi-metro cell architecture with global control plane. - Tighter SLO targets and operational levers to hit them. - Event surge pre-warm and adaptive geohash partitioning. - Stronger reservation consistency via ownership hashing and sagas. - Privacy and retention policies for cross-metro deployments.